Quiz Practice Problems (Transactions and Locks)

Instructions: For each problem, you must perform all of the following tasks correctly in order to get credit.

1. Circle all SQL commands that block.
2. Write an X through all commands that error.
3. If a command causes a deadlock, write `DEADLOCK` and stop the problem.
4. Write the output of each `SELECT` statement.

You will be allowed to use a computer and access the internet, but you will not be allowed to connect to postgres and run postgres commands.

1 Basic Transactions

Note 1. SQL transactions are started by the BEGIN command and terminate with either COMMIT (indicating success) or ABORT / ROLLBACK (indicating failure). Transactions are atomic (either everything works or everything fails) and they are isolated (effects of transactions are not visible until COMMIT).

Problem 1:

Session 1

```
1  CREATE TABLE t ( a INT );
2
3  INSERT INTO t VALUES (5);
4  INSERT INTO t VALUES (6);
5
6  INSERT INTO t VALUES (2);
```

Session 2

```
1
2  SELECT count(*) FROM t;
3
4
5  SELECT count(*) FROM t;
6
7  SELECT count(*) FROM t;
```

Problem 2:

Session 1

```
1  CREATE TABLE t ( a INT );
2
3  BEGIN;
4  INSERT INTO t VALUES (5);
5  INSERT INTO t VALUES (6);
6
7  INSERT INTO t VALUES (2);
8  COMMIT;
```

Session 2

```
1
2  SELECT count(*) FROM t;
3
4
5
6  SELECT count(*) FROM t;
7
8
9  SELECT count(*) FROM t;
```

Problem 3:

Session 1

```
1  CREATE TABLE t ( a INT );
2
3  BEGIN;
4  INSERT INTO t VALUES (5);
5  INSERT INTO t VALUES (6);
6
7  INSERT INTO t VALUES (2);
8  ROLLBACK;
```

Session 2

```
1
2  SELECT count(*) FROM t;
3
4
5
6  SELECT count(*) FROM t;
7
8
9  SELECT count(*) FROM t;
```

Problem 4:

Session 1

```
1  CREATE TABLE t ( a INT );
2
3  BEGIN;
4  INSERT INTO t VALUES (5);
5  INSERT INTO t VALUES (6);
6
7  INSERT INTO t VALUES (2);
8  ABORT;
```

Session 2

```
1
2  SELECT count(*) FROM t;
3
4
5
6  SELECT count(*) FROM t;
7
8
9  SELECT count(*) FROM t;
```

1.1 Isolation Levels

Note 2. A transaction will never effect concurrent processes until COMMIT. But isolation levels determine the "state" of the database within a transaction.

Problem 5:

Session 1

```
1  CREATE TABLE t ( a INT );
2
3
4  BEGIN;
5  INSERT INTO t VALUES (5);
6  INSERT INTO t VALUES (6);
7
8  INSERT INTO t VALUES (2);
9  COMMIT;
```

Session 2

```
1
2  BEGIN;
3  SELECT count(*) FROM t;
4
5
6
7  SELECT count(*) FROM t;
8
9
10 SELECT count(*) FROM t;
11 COMMIT;
12 SELECT count(*) FROM t;
```

Problem 6:

Session 1

```
1  CREATE TABLE t ( a INT );
2
3
4
5  BEGIN;
6  INSERT INTO t VALUES (5);
7  INSERT INTO t VALUES (6);
8
9  INSERT INTO t VALUES (2);
10 COMMIT;
```

Session 2

```
1
2  BEGIN ISOLATION LEVEL
3  REPEATABLE READ;
4  SELECT count(*) FROM t;
5
6
7
8  SELECT count(*) FROM t;
9
10
11 SELECT count(*) FROM t;
12 COMMIT;
13 SELECT count(*) FROM t;
```

Problem 7:

Session 1

```
1  CREATE TABLE t ( a INT );
2
3
4  BEGIN;
5  INSERT INTO t VALUES (5);
6  INSERT INTO t VALUES (6);
7
8  INSERT INTO t VALUES (2);
9  COMMIT;
```

Session 2

```
1
2  BEGIN ISOLATION LEVEL
3  READ COMMITTED;
4
5
6
7  SELECT count(*) FROM t;
8
9
10 SELECT count(*) FROM t;
11 COMMIT;
12 SELECT count(*) FROM t;
```

Note 3. From here on out, you should understand all of the examples as if any transaction was in any isolation level.

Problem 8:

Session 1

```
1  CREATE TABLE t ( a INT );
2
3
4  BEGIN;
5  INSERT INTO t VALUES (5);
6  INSERT INTO t VALUES (6);
7  INSERT INTO t VALUES (2);
8  COMMIT;
```

Session 2

```
1
2  BEGIN ISOLATION LEVEL
3  REPEATABLE READ;
4
5
6
7
8
9  SELECT count(*) FROM t;
10 COMMIT;
11 SELECT count(*) FROM t;
```

Problem 9:

Session 1

```
1  CREATE TABLE t ( a INT );
2
3
4
5  BEGIN ISOLATION LEVEL
6  REPEATABLE READ;
7  INSERT INTO t VALUES (5);
8  SELECT count(*) FROM t;
9  INSERT INTO t VALUES (6);
10 SELECT count(*) FROM t;
11 INSERT INTO t VALUES (2);
12 SELECT count(*) FROM t;
13 COMMIT;
```

Session 2

```
1
2  BEGIN ISOLATION LEVEL
3  REPEATABLE READ;
4  SELECT count(*) FROM t;
5
6
7
8
9
10
11
12
13
14 SELECT count(*) FROM t;
15 COMMIT;
16 SELECT count(*) FROM t;
```

2 Explicit Locks

Note 4. Locks are used to manage concurrency and ensure that multiple processes do not perform incompatible actions at the same time. Processes can *block* when they try to acquire a lock on a table that is already locked. A blocked process waits for the previous process to unblock before proceeding.

Problem 10:

Session 1

```
1  CREATE TABLE t ( a INT );
2  BEGIN;
3  LOCK TABLE t IN EXCLUSIVE MODE;
4
5
6  COMMIT;
```

Session 2

```
1
2
3
4  BEGIN;
5  LOCK TABLE t IN SHARE MODE;
6
7  COMMIT;
```

Problem 11:

Session 1

```
1  CREATE TABLE t ( a INT );
2  BEGIN;
3  LOCK TABLE t IN SHARE MODE;
4
5
6  COMMIT;
```

Session 2

```
1
2
3
4  BEGIN;
5  LOCK TABLE t IN EXCLUSIVE MODE;
6
7  COMMIT;
```

Problem 12:

Session 1

```
1  CREATE TABLE t ( a INT );
2  BEGIN;
3  LOCK TABLE t IN SHARE MODE;
4
5
6  COMMIT;
```

Session 2

```
1
2
3
4  BEGIN;
5  LOCK TABLE t IN ROW SHARE MODE;
6
7  COMMIT;
```

Problem 13:

Session 1

```
1  CREATE TABLE t ( a INT );
2  BEGIN;
3  LOCK TABLE t IN EXCLUSIVE MODE;
4  COMMIT;
```

Session 2

```
1
2
3
4
5  BEGIN;
6  LOCK TABLE t IN EXCLUSIVE MODE;
7  COMMIT;
```

Problem 14:

Session 1

```
1  CREATE TABLE t ( a INT );
2  BEGIN;
3  LOCK TABLE t IN ROW EXCLUSIVE MODE;
4
5
6
7  COMMIT;
```

Session 2

```
1
2
3
4  BEGIN;
5  LOCK TABLE t IN ROW SHARE MODE;
6  COMMIT;
```

3 Deadlocks

Note 5. If two processes are each waiting on each other, this is a *deadlock*. When postgres detects a deadlock, it will automatically terminate one of the processes. The choice of which process is non-deterministic.

Problem 15:

Session 1

```
1  CREATE TABLE t ( a INT );
2
3  BEGIN;
4  LOCK TABLE t IN EXCLUSIVE MODE;
5
6
7  LOCK TABLE u IN EXCLUSIVE MODE;
8
9  COMMIT;
```

Session 2

```
1
2  CREATE TABLE u ( a INT );
3
4
5  BEGIN;
6  LOCK TABLE u IN EXCLUSIVE MODE;
7
8  LOCK TABLE t IN EXCLUSIVE MODE;
9
10 COMMIT;
```

Problem 16:

Session 1

```
1  CREATE TABLE t ( a INT );
2
3  BEGIN;
4  LOCK TABLE t IN SHARE MODE;
5
6
7  LOCK TABLE u IN SHARE MODE;
8
9  COMMIT;
```

Session 2

```
1
2  CREATE TABLE u ( a INT );
3
4
5  BEGIN;
6  LOCK TABLE u IN SHARE MODE;
7
8  LOCK TABLE t IN SHARE MODE;
9
10 COMMIT;
```

Problem 17:

Session 1

```
1  CREATE TABLE t ( a INT );
2
3  BEGIN;
4  LOCK TABLE t IN SHARE MODE;
5
6
7  LOCK TABLE u IN ROW EXCLUSIVE MODE;
8
9  COMMIT;
```

Session 2

```
1
2  CREATE TABLE u ( a INT );
3
4
5  BEGIN;
6  LOCK TABLE u IN EXCLUSIVE MODE;
7
8  LOCK TABLE t IN ROW SHARE MODE;
9
10 COMMIT;
```

Problem 18:

Session 1

```
1  CREATE TABLE t ( a INT );
2
3  BEGIN;
4  LOCK TABLE t IN SHARE MODE;
5  LOCK TABLE u IN ROW EXCLUSIVE MODE;
6  COMMIT;
```

Session 2

```
1
2  CREATE TABLE u ( a INT );
3
4
5
6
7  BEGIN;
8  LOCK TABLE u IN EXCLUSIVE MODE;
9  LOCK TABLE t IN ROW SHARE MODE;
10 COMMIT;
```

Problem 19:

Session 1

```
1  CREATE TABLE t ( a INT );
2
3  BEGIN;
4  LOCK TABLE t IN SHARE MODE;
5  LOCK TABLE u IN ROW EXCLUSIVE MODE;
6
7
8
9  COMMIT;
```

Session 2

```
1
2  CREATE TABLE u ( a INT );
3
4
5
6  BEGIN;
7  LOCK TABLE u IN EXCLUSIVE MODE;
8  LOCK TABLE t IN ROW SHARE MODE;
9
10 COMMIT;
```

Problem 20:

Session 1

```
1  CREATE TABLE t ( a INT );
2
3  BEGIN;
4
5  LOCK TABLE t IN ACCESS SHARE MODE;
6
7
8  LOCK TABLE u IN ROW EXCLUSIVE MODE;
9  COMMIT;
```

Session 2

```
1
2  CREATE TABLE u ( a INT );
3
4  BEGIN;
5
6  LOCK TABLE u IN EXCLUSIVE MODE;
7  LOCK TABLE t IN SHARE MODE;
8
9
10 COMMIT;
```

4 Implicit Locks I

Note 6. All SQL commands acquire an *implicit* lock. In order to determine whether the command will block, we must understand whether the corresponding explicit lock would block.

Problem 21:

Session 1	Session 2
1 CREATE TABLE t (a INT);	1
2 BEGIN;	2
3 LOCK TABLE t IN ACCESS EXCLUSIVE MODE;	3
4	4 INSERT INTO t VALUES (5);
5 COMMIT;	

Problem 22:

Session 1	Session 2
1 CREATE TABLE t (a INT);	1
2 BEGIN;	2
3 LOCK TABLE t IN ACCESS EXCLUSIVE MODE;	3
4	4 SELECT count(*) FROM t;
5 COMMIT;	

Problem 23:

Session 1	Session 2
1 CREATE TABLE t (a INT);	1
2 BEGIN;	2
3 LOCK TABLE t IN SHARE MODE;	3
4	4 INSERT INTO t VALUES (1);
5 COMMIT;	

Problem 24:

Session 1	Session 2
1 CREATE TABLE t (a INT);	1
2 BEGIN;	2
3 LOCK TABLE t IN SHARE MODE;	3
4	4 SELECT count(*) FROM t;
5 COMMIT;	

Problem 25:

Session 1	Session 2
1 CREATE TABLE t (a INT);	1
2 BEGIN;	2
3 LOCK TABLE t IN ROW SHARE MODE;	3
4	4 INSERT INTO t VALUES (1);
5 COMMIT;	

4.1 More interesting commands

Note 7. This section contains many new commands that we have not yet seen in class. You are still responsible for being able to determine whether the command will block or not.

Problem 26:

Session 1	Session 2
1 CREATE TABLE t (a INT);	1
2 BEGIN;	2
3 LOCK TABLE t IN ROW EXCLUSIVE MODE;	3
4	4 CREATE INDEX ON t(a);
5 COMMIT;	

Problem 27:

Session 1	Session 2
1 CREATE TABLE t (a INT);	1
2 BEGIN;	2
3 LOCK TABLE t IN SHARE MODE;	3
4	4 VACUUM t;
5 COMMIT;	

Problem 28:

Session 1	Session 2
1 CREATE TABLE t (a INT);	1
2 BEGIN;	2
3 LOCK TABLE t IN ROW SHARE MODE;	3
4	4 VACUUM FULL t;
5 COMMIT;	

Problem 29:

Session 1	Session 2
1 CREATE TABLE t (a INT);	1
2 BEGIN;	2
3 LOCK TABLE t IN ROW EXCLUSIVE MODE;	3
4	4 ANALYZE t;
5 COMMIT;	

Problem 30:

Session 1	Session 2
1 CREATE TABLE t (a INT);	1
2 BEGIN;	2
3 LOCK TABLE t IN ROW EXCLUSIVE MODE;	3
4	4 CLUSTER t;
5 COMMIT;	

Problem 31:

Session 1	Session 2
1 CREATE TABLE t (a INT);	1
2 BEGIN;	2
3 INSERT INTO t VALUES (5);	3
4	4 BEGIN;
5	5 INSERT INTO t VALUES (6);
6 COMMIT;	6
	7 COMMIT;

5 Row Level Locks

Note 8. Most commands that affect individual rows acquire locks only on those rows. This will allow many processes to run INSERT commands, for example, without blocking each other. Understanding how these row level locks interoperate is important for efficient code to parallel insert into a database.

Problem 32:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (1);
3  INSERT INTO t VALUES (2);
4  INSERT INTO t VALUES (3);
5
6  BEGIN;
7  UPDATE t SET a=6 WHERE a=1;
8
9
10 COMMIT;
```

Session 2

```
1
2
3
4
5  BEGIN;
6
7
8  UPDATE t SET a=7 WHERE a=1;
9  COMMIT;
```

Problem 33:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (1);
3  INSERT INTO t VALUES (2);
4  INSERT INTO t VALUES (3);
5
6  BEGIN;
7  UPDATE t SET a=6 WHERE a=1;
8
9
10 COMMIT;
```

Session 2

```
1
2
3
4
5  BEGIN;
6
7
8  DELETE FROM t WHERE a=2;
9  COMMIT;
```

Problem 34:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (1);
3  INSERT INTO t VALUES (2);
4  INSERT INTO t VALUES (3);
5
6  BEGIN;
7  UPDATE t SET a=6 WHERE a=1;
8
9
10 COMMIT;
```

Session 2

```
1
2
3
4
5  BEGIN;
6
7
8  DELETE FROM t WHERE a=1;
9  COMMIT;
```

Problem 35:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (1);
3  INSERT INTO t VALUES (2);
4  INSERT INTO t VALUES (3);
5
6  BEGIN;
7  UPDATE t SET a=6 WHERE a=1;
8
9
10 COMMIT;
```

Session 2

```
1
2
3
4
5  BEGIN;
6
7
8  DELETE FROM t;
9  COMMIT;
```

Problem 36:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (1);
3  INSERT INTO t VALUES (2);
4  INSERT INTO t VALUES (3);
5
6  BEGIN;
7  DELETE FROM t WHERE a=3;
8
9
10 COMMIT;
```

Session 2

```
1
2
3
4
5  BEGIN;
6
7
8  UPDATE t SET a=5;
9  COMMIT;
```

Problem 37:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (1);
3  INSERT INTO t VALUES (2);
4  INSERT INTO t VALUES (3);
5
6  BEGIN;
7  DELETE FROM t WHERE a=4;
8
9
10 COMMIT;
```

Session 2

```
1
2
3
4
5  BEGIN;
6
7
8  UPDATE t SET a=5;
9  COMMIT;
```

Problem 38:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (1);
3  INSERT INTO t VALUES (2);
4  INSERT INTO t VALUES (3);
5
6  BEGIN;
7  UPDATE t SET a=6 WHERE a=1;
8
9
10 COMMIT;
```

Session 2

```
1
2
3
4
5  BEGIN;
6
7
8  UPDATE t SET a=7 WHERE a=2;
9  COMMIT;
```

Problem 39:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (1);
3  INSERT INTO t VALUES (2);
4  INSERT INTO t VALUES (3);
5
6  BEGIN;
7  SELECT * FROM t WHERE a=2 FOR UPDATE;
8
9
10 COMMIT;
```

Session 2

```
1
2
3
4
5  BEGIN;
6
7
8  UPDATE t SET a=7 WHERE a=2;
9  COMMIT;
```

Problem 40:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (1);
3  INSERT INTO t VALUES (2);
4  INSERT INTO t VALUES (3);
5
6  BEGIN;
7  SELECT * FROM t WHERE a=2 FOR UPDATE;
8
9
10 COMMIT;
```

Session 2

```
1
2
3
4
5  BEGIN;
6
7
8  UPDATE t SET a=7 WHERE a=3;
9  COMMIT;
```

Problem 41:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (1);
3  INSERT INTO t VALUES (2);
4  INSERT INTO t VALUES (3);
5
6  BEGIN;
7  SELECT * FROM t FOR UPDATE;
8
9
10 COMMIT;
```

Session 2

```
1
2
3
4
5  BEGIN;
6
7
8  UPDATE t SET a=7 WHERE a=3;
9  COMMIT;
```

Problem 42:

Session 1

```
1  CREATE TABLE t ( a INT  UNIQUE );
2  INSERT INTO t VALUES (9);
3  INSERT INTO t VALUES (10);
4  BEGIN;
5  DELETE FROM t WHERE a=9;
6
7
8
9  INSERT INTO t VALUES (9);
10 DELETE FROM t;
11
12 ABORT;
```

Session 2

```
1
2
3
4
5
6  BEGIN;
7  UPDATE t SET a=11 WHERE a=9;
8  DELETE FROM t;
9
10
11 INSERT INTO t VALUES (8);
12
13 INSERT INTO t VALUES (9);
14 COMMIT;
15 SELECT count(*) FROM t;
```

Problem 43:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (9);
3  INSERT INTO t VALUES (10);
4  BEGIN;
5  DELETE FROM t WHERE a=9;
6
7
8
9  INSERT INTO t VALUES (9);
10 DELETE FROM t;
11
12 ABORT;
```

Session 2

```
1
2
3
4
5
6  BEGIN;
7  UPDATE t SET a=11 WHERE a=9;
8  DELETE FROM t;
9
10
11 INSERT INTO t VALUES (8);
12
13 INSERT INTO t VALUES (9);
14 COMMIT;
15 SELECT count(*) FROM t;
```

6 Unique constraints

Note 9. A non-NULL insert/update into a UNIQUE column takes an exclusive row-level lock on the index entry. NULLs never conflict.

Problem 44:

Session 1

```
1  CREATE TABLE t ( a INT UNIQUE );
2  BEGIN;
3  INSERT INTO t VALUES (5);
4
5
6  COMMIT;
```

Session 2

```
1
2
3
4  BEGIN;
5  INSERT INTO t VALUES (6);
6
7  COMMIT;
```

Problem 45:

Session 1	Session 2
1 CREATE TABLE t (a INT UNIQUE);	1
2 BEGIN;	2
3 INSERT INTO t VALUES (NULL);	3
4	4 BEGIN;
5	5 INSERT INTO t VALUES (6);
6 COMMIT;	6
	7 COMMIT;

Problem 46:

Session 1	Session 2
1 CREATE TABLE t (a INT UNIQUE);	1
2 BEGIN;	2
3 INSERT INTO t VALUES (5);	3
4	4 BEGIN;
5	5 INSERT INTO t VALUES (NULL);
6 COMMIT;	6
	7 COMMIT;

Problem 47:

Session 1	Session 2
1 CREATE TABLE t (a INT UNIQUE);	1
2	2 CREATE TABLE u (a INT UNIQUE);
3 BEGIN;	3
4 INSERT INTO t VALUES (NULL);	4
5	5 BEGIN;
6	6 INSERT INTO u VALUES (NULL);
7 INSERT INTO u VALUES (6);	7
8	8 INSERT INTO t VALUES (8);
9	9 COMMIT;
10 COMMIT;	

Problem 48:

Session 1

```
1  CREATE TABLE t ( a INT UNIQUE );
2
3  BEGIN;
4  INSERT INTO t VALUES (5);
5
6
7  INSERT INTO u VALUES (NULL);
8
9
10 COMMIT;
```

Session 2

```
1
2  CREATE TABLE u ( a INT UNIQUE );
3
4
5  BEGIN;
6  INSERT INTO u VALUES (7);
7
8  INSERT INTO t VALUES (8);
9  COMMIT;
```

Problem 49:

Session 1

```
1  CREATE TABLE t ( a INT UNIQUE );
2
3  BEGIN;
4  INSERT INTO t VALUES (5);
5
6
7  INSERT INTO u VALUES (6);
8
9
10 COMMIT;
```

Session 2

```
1
2  CREATE TABLE u ( a INT UNIQUE );
3
4
5  BEGIN;
6  INSERT INTO u VALUES (7);
7
8  INSERT INTO t VALUES (NULL);
9  COMMIT;
```

Problem 50:

Session 1

```
1  CREATE TABLE t(a INT UNIQUE);
2  CREATE TABLE u(
3      b INT REFERENCES t(a)
4      DEFERRABLE INITIALLY DEFERRED
5      );
6  BEGIN;
7
8  INSERT INTO t VALUES (9);
9
10 COMMIT;
```

Session 2

```
1
2
3
4
5
6  BEGIN;
7
8  SELECT count(*) FROM u;
9
10 INSERT INTO u VALUES (9);
11 COMMIT;
```

Problem 51:

Session 1

```
1  CREATE TABLE t(a INT UNIQUE);
2  CREATE TABLE u(
3      b INT REFERENCES t(a)
4      DEFERRABLE INITIALLY DEFERRED
5      );
6  BEGIN;
7
8
9  INSERT INTO t VALUES (9);
10
11 COMMIT;
```

Session 2

```
1
2
3
4
5
6  BEGIN ISOLATION LEVEL
7  REPEATABLE READ;
8
9  SELECT count(*) FROM u;
10
11 INSERT INTO u VALUES (9);
12 COMMIT;
```

Problem 52:

Session 1

```
1  CREATE TABLE t(
2      a INT UNIQUE
3      DEFERRABLE INITIALLY DEFERRED
4      );
5  CREATE TABLE u(b INT REFERENCES t(a));
6  BEGIN;
7
8  INSERT INTO t VALUES (9);
9
10 COMMIT;
```

Session 2

```
1
2
3
4
5
6  BEGIN;
7
8  SELECT count(*) FROM u;
9
10 INSERT INTO u VALUES (9);
11 COMMIT;
```

Problem 53:

Session 1

```
1  CREATE TABLE t(
2      a INT UNIQUE
3      DEFERRABLE INITIALLY DEFERRED
4      );
5  CREATE TABLE u(b INT REFERENCES t(a));
6  BEGIN;
7
8
9  INSERT INTO t VALUES (9);
10
11 COMMIT;
```

Session 2

```
1
2
3
4
5
6  BEGIN ISOLATION LEVEL
7  REPEATABLE READ;
8
9  SELECT count(*) FROM u;
10
11 INSERT INTO u VALUES (9);
12 COMMIT;
```

Problem 54:

Session 1

```
1  CREATE TABLE t ( a INT  UNIQUE );
2  BEGIN;
3  INSERT INTO t VALUES (9);
4  INSERT INTO t VALUES (10);
5
6
7  INSERT INTO t VALUES (8);
8
9
10
11 ABORT;
```

Session 2

```
1
2
3
4
5  BEGIN;
6  INSERT INTO t VALUES (8);
7
8  INSERT INTO t VALUES (9);
9  COMMIT;
10 SELECT count(*) FROM t;
```

7 Interaction between Isolation Levels and Row Level locks

Note 10. Postgres always evaluates WHERE clauses twice: once before acquiring a lock and once after the lock is acquired (the other session releases the lock and this session acquires it). This can cause us to miss entries. Under REPEATABLE READ, if the row changed since the snapshot, it raises a serialization failure.

Problem 55:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (9);
3  INSERT INTO t VALUES (10);
4  BEGIN;
5
6  UPDATE t SET a = a+1;
7
8
9  COMMIT;
```

Session 2

```
1
2
3
4
5  BEGIN;
6
7  DELETE FROM t WHERE a=10;
8  COMMIT;
9
10 SELECT count(*) FROM t;
```

Problem 56:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (9);
3  INSERT INTO t VALUES (10);
4  BEGIN;
5
6  UPDATE t SET a = a+1;
7
8
9  ABORT;
```

Session 2

```
1
2
3
4
5  BEGIN;
6
7  DELETE FROM t WHERE a=10;
8  COMMIT;
9  SELECT count(*) FROM t;
```

Problem 57:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (9);
3  INSERT INTO t VALUES (10);
4  BEGIN;
5
6
7  UPDATE t SET a = a+1;
8
9
10 COMMIT;
```

Session 2

```
1
2
3
4
5  BEGIN ISOLATION LEVEL
6  REPEATABLE READ;
7
8  DELETE FROM t WHERE a=10;
9
10 COMMIT;
```

Problem 58:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (9);
3  INSERT INTO t VALUES (10);
4  BEGIN;
5
6
7  UPDATE t SET a = a+1;
8
9
10 ABORT;
```

Session 2

```
1
2
3
4
5  BEGIN ISOLATION LEVEL
6  REPEATABLE READ;
7
8  DELETE FROM t WHERE a=10;
9  COMMIT;
```

Problem 59:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (9);
3  INSERT INTO t VALUES (10);
4
5  UPDATE t SET a = a+1;
```

Session 2

```
1
2
3
4  BEGIN;
5
6  DELETE FROM t WHERE a=10;
7  COMMIT;
8  SELECT count(*) FROM t;
```

Problem 60:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (9);
3  INSERT INTO t VALUES (10);
4
5
6  UPDATE t SET a = a+1;
```

Session 2

```
1
2
3
4  BEGIN;
5  SELECT count(*) FROM  t;
6
7  DELETE FROM t WHERE a=10;
8  COMMIT;
9  SELECT count(*) FROM t;
```

Problem 61:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (9);
3  INSERT INTO t VALUES (10);
4
5
6
7  UPDATE t SET a = a+1;
```

Session 2

```
1
2
3
4  BEGIN ISOLATION LEVEL
5  READ COMMITTED;
6  SELECT count(*) FROM  t;
7
8  DELETE FROM t WHERE a=10;
9  COMMIT;
10 SELECT count(*) FROM t;
```

Problem 62:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (9);
3  INSERT INTO t VALUES (10);
4
5
6
7  UPDATE t SET a = a+1;
```

Session 2

```
1
2
3
4  BEGIN ISOLATION LEVEL
5  READ UNCOMMITTED;
6  SELECT count(*) FROM  t;
7
8  DELETE FROM t WHERE a=10;
9  COMMIT;
10 SELECT count(*) FROM t;
```

Problem 63:

Session 1

```
1  CREATE TABLE t ( a INT );
2  INSERT INTO t VALUES (9);
3  INSERT INTO t VALUES (10);
4
5
6
7  UPDATE t SET a = a+1;
```

Session 2

```
1
2
3
4  BEGIN ISOLATION LEVEL
5  REPEATABLE READ;
6  SELECT count(*) FROM  t;
7
8  DELETE FROM t WHERE a=10;
9  COMMIT;
10 SELECT count(*) FROM t;
```

8 Foreign Keys

Problem 64:

Session 1

```
1  CREATE TABLE t(a INT UNIQUE);
2  CREATE TABLE u(b INT REFERENCES t(a));
3  BEGIN;
4
5  INSERT INTO t VALUES (9);
6
7  COMMIT;
```

Session 2

```
1
2
3
4  BEGIN;
5
6  INSERT INTO u VALUES (9);
7
8  COMMIT;
```

Problem 65:

Session 1

```
1  CREATE TABLE t(a INT UNIQUE);
2  CREATE TABLE u(b INT REFERENCES t(a));
3
4  INSERT INTO t VALUES (9);
```

Session 2

```
1
2
3  BEGIN;
4
5  INSERT INTO u VALUES (9);
6  COMMIT;
```

Problem 66:

Session 1

```
1  CREATE TABLE t(a INT UNIQUE);
2  CREATE TABLE u(b INT REFERENCES t(a));
3  INSERT INTO t VALUES (8);
4  INSERT INTO t VALUES (9);
5  BEGIN;
6
7  INSERT INTO u VALUES (9);
8
9  COMMIT;
```

Session 2

```
1
2
3
4
5
6  BEGIN;
7
8  DELETE FROM t WHERE a=9;
9
10 COMMIT;
```

Problem 67:

Session 1

```
1  CREATE TABLE t(a INT UNIQUE);
2  CREATE TABLE u(b INT REFERENCES t(a));
3  INSERT INTO t VALUES (8);
4  INSERT INTO t VALUES (9);
5  BEGIN;
6
7  INSERT INTO u VALUES (9);
8
9  COMMIT;
```

Session 2

```
1
2
3
4
5
6  BEGIN;
7
8  DELETE FROM t WHERE a=8;
9
10 COMMIT;
```

Problem 68:

Session 1

```
1  CREATE TABLE t(a INT UNIQUE);
2  CREATE TABLE u(b INT REFERENCES t(a));
3  INSERT INTO t VALUES (8);
4  INSERT INTO t VALUES (9);
5  BEGIN;
6
7  INSERT INTO u VALUES (9);
8
9  ABORT;
```

Session 2

```
1
2
3
4
5
6  BEGIN;
7
8  DELETE FROM t WHERE a=9;
9
10 COMMIT;
```

Problem 69:

Session 1

```
1  CREATE TABLE t(a INT UNIQUE);
2  CREATE TABLE u(b INT REFERENCES t(a));
3  INSERT INTO t VALUES (8);
4  INSERT INTO t VALUES (9);
5  BEGIN;
6
7  INSERT INTO u VALUES (9);
8
9  ABORT;
```

Session 2

```
1
2
3
4
5
6  BEGIN;
7
8  DELETE FROM t WHERE a=8;
9
10 COMMIT;
```

Problem 70:

Session 1

```
1  CREATE TABLE t(a INT UNIQUE);
2  CREATE TABLE u(b INT REFERENCES t(a));
3  INSERT INTO t VALUES (8);
4  INSERT INTO t VALUES (9);
5  BEGIN;
6
7
8  INSERT INTO u VALUES (9);
9
10 COMMIT;
```

Session 2

```
1
2
3
4
5
6  BEGIN;
7  DELETE FROM t WHERE a=9;
8
9  COMMIT;
```

Problem 71:

Session 1

```
1  CREATE TABLE t(a INT UNIQUE);
2  CREATE TABLE u(b INT REFERENCES t(a));
3  INSERT INTO t VALUES (8);
4  INSERT INTO t VALUES (9);
5  BEGIN;
6
7
8  INSERT INTO u VALUES (9);
9
10 COMMIT;
```

Session 2

```
1
2
3
4
5
6  BEGIN;
7  DELETE FROM t WHERE a=8;
8
9  COMMIT;
```

Problem 72:

Session 1

```
1  CREATE TABLE t(a INT UNIQUE);
2  CREATE TABLE u(b INT REFERENCES t(a));
3  BEGIN;
4
5  INSERT INTO t VALUES (9);
6
7  COMMIT;
```

Session 2

```
1
2
3
4  BEGIN;
5
6  SELECT count(*) FROM u;
7
8  INSERT INTO u VALUES (9);
9  COMMIT;
```

Problem 73:

Session 1

```
1  CREATE TABLE t(a INT UNIQUE);
2  CREATE TABLE u(b INT REFERENCES t(a));
3  BEGIN;
4
5
6  INSERT INTO t VALUES (9);
7
8  COMMIT;
```

Session 2

```
1
2
3
4  BEGIN ISOLATION LEVEL
5  REPEATABLE READ;
6
7  SELECT count(*) FROM u;
8
9  INSERT INTO u VALUES (9);
10 COMMIT;
```

9 Deferring Constraints

Note 11. By default, postgres will check constraints at the time that data is inserted. Adding the DEFERRABLE INITIALLY DEFERRED line to the constraint causes the constraint to be checked at time of COMMIT. Sometimes this is necessary for correctness, and sometimes it is necessary for speed.

Note 12. For each problem, you should consider what would happen both with and without the DEFERRABLE INITIALLY DEFERRED line.

Problem 74:

Session 1

```
1  CREATE TABLE t (
2      a INT UNIQUE
3      DEFERRABLE INITIALLY DEFERRED
4      );
5  BEGIN;
6  INSERT INTO t VALUES (8);
7  INSERT INTO t VALUES (8);
8  COMMIT;
```

Session 2

Problem 75:

Session 1

```
1  CREATE TABLE t (a INT UNIQUE);
2  CREATE TABLE u (
3      b INT REFERENCES t(a)
4      DEFERRABLE INITIALLY DEFERRED
5      );
6  BEGIN;
7
8
9  INSERT INTO t VALUES (8);
10 COMMIT;
```

Session 2

```
1
2
3
4
5
6
7  BEGIN;
8  INSERT INTO u VALUES (8);
9
10
11 COMMIT;
```

Problem 76:

Session 1

```
1  CREATE TABLE t (a INT UNIQUE);
2  CREATE TABLE u (
3      b INT REFERENCES t(a)
4      DEFERRABLE INITIALLY DEFERRED
5      );
6  BEGIN;
7
8
9  INSERT INTO t VALUES (8);
10 ABORT;
```

Session 2

```
1
2
3
4
5
6
7  BEGIN;
8  INSERT INTO u VALUES (8);
9
10
11 COMMIT;
```

Problem 77:

Session 1

```
1  CREATE TABLE t (a INT UNIQUE);
2  CREATE TABLE u (
3      b INT REFERENCES t(a)
4      DEFERRABLE INITIALLY DEFERRED
5      );
6  BEGIN;
7
8
9  LOCK TABLE t IN EXCLUSIVE MODE;
10
11 INSERT INTO t VALUES (8);
12 COMMIT;
```

Session 2

```
1
2
3
4
5
6
7  BEGIN;
8  INSERT INTO u VALUES (8);
9
10 COMMIT;
```

Problem 78:

Session 1

```
1  CREATE TABLE t (a INT UNIQUE);
2  CREATE TABLE u (
3      b INT REFERENCES t(a)
4      DEFERRABLE INITIALLY DEFERRED
5      );
6  BEGIN;
7
8
9  LOCK TABLE t IN EXCLUSIVE MODE;
10
11 COMMIT;
```

Session 2

```
1
2
3
4
5
6
7  BEGIN;
8  INSERT INTO u VALUES (8);
9
10 COMMIT;
```

10 Everything at Once

Note 13. The problems below test the interaction of all the previous knowledge. These problems are harder because there are no "hints" about what types of effects you should be looking for. Realworld problems are significantly more complicated than these problems.

Problem 79:

Session 1

```
1  CREATE TABLE t (a INT PRIMARY KEY);
2
3
4
5  BEGIN;
6  INSERT INTO t VALUES (1);
7  INSERT INTO t VALUES (2);
8  INSERT INTO u VALUES (2);
9
10
11
12 LOCK TABLE t IN EXCLUSIVE MODE;
13 SELECT count(*) FROM t;
14
15
16 INSERT INTO t VALUES (NULL);
17 COMMIT;
18
19
20
21 SELECT count(*) FROM u;
```

Session 2

```
1
2  CREATE TABLE u (
3      b INT REFERENCES t(a)
4      );
5
6
7
8
9  BEGIN ISOLATION LEVEL
10 REPEATABLE READ;
11 SELECT count(*) FROM u;
12
13
14 INSERT INTO u VALUES (1);
15 SELECT count(*) FROM u;
16
17
18 INSERT INTO u VALUES (NULL);
19 COMMIT;
20 SELECT count(*) FROM t;
```

Problem 80:

Session 1

```
1  CREATE TABLE t (a INT PRIMARY KEY);
2
3  BEGIN;
4  INSERT INTO t VALUES (1);
5  INSERT INTO t VALUES (2);
6  INSERT INTO u VALUES (2);
7
8
9
10 LOCK TABLE u IN ROW EXCLUSIVE MODE;
11 SELECT count(*) FROM t;
12
13
14 INSERT INTO t VALUES (3);
15 COMMIT;
16
17
18
19 SELECT count(*) FROM t;
```

Session 2

```
1
2  CREATE TABLE u (b INT NOT NULL);
3
4
5
6
7  BEGIN ISOLATION LEVEL
8  REPEATABLE READ;
9  INSERT INTO t VALUES (2);
10
11
12 INSERT INTO u VALUES (1);
13 SELECT count(*) FROM u;
14
15
16 UPDATE u SET b=5;
17 COMMIT;
18 SELECT count(*) FROM t;
```